数独与整数规划

把填数字变成一组开关

数学建模笔记

2026 年 10 月

这篇文章想做的事

做数独时，我们会在空格里写候选数，圈出某个只能填 7 的位置，再沿着行、列和小宫格一路排除。计算机也能这么做，不过还有一条很有意思的路：**不教它任何数独技巧，只把规则写成方程。**

这篇文章从一张四阶小数独出发，讲清楚三个问题：为什么一个格子要拆成好几个 0–1 变量；为什么没有“最大”或“最小”的数独，也能交给整数规划求解；以及，找到一个答案之后，怎样确认它是**唯一**的答案。最后再看看线性松弛、精确覆盖和数独变体，把这个小模型往外推几步。（本文由 GPT 写作）

1　引子：数独到底在要求什么

标准数独有 9×9 个格子，每格填 1 到 9 中的一个数。每一行、每一列、每个 3×3 小宫格里，九个数字都恰好出现一次，题目预先给出的数字不能改。规则几句话就讲完了，难的却是把空格补全。

为了能用纸笔追踪，我们先把它缩成**四阶数独**：4×4 个格子，数字换成 $1,2,3,4$，小宫格换成 2×2。下面左边是题目，右边是它的一个解，点表示空格。

1	·	·	4
·	4	1	·
2	·	4	·
·	3	·	1

$\longrightarrow$

1	2	3	4
3	4	1	2
2	1	4	3
4	3	2	1

第一行缺 $2,3$；第二列已经有 $4,3$，所以第一行第二列只能填 2。接着第一行第三列就只能填 3。这是一段熟悉的数独推理。

但我们接下来要做的，是把“只能”这两个字翻译成约束，让这些推理从同一套模型里自然生长出来。

2　准备：整数规划里有什么

一个线性整数规划通常写成

$$\min\ \boldsymbol{c}^{\mathsf{T}}\boldsymbol{x},\qquad A\boldsymbol{x}\le\boldsymbol{b},\quad \boldsymbol{x}\in\mathbb{Z}^{m},$$

也可以有等式和变量上下界。这里需要区分三件东西：

- **变量**：哪些决定还没有做，例如一个位置填什么；
- **约束**：哪些决定可以同时成立，例如同一行不能重复；
- **目标函数**：在允许的决定中，偏爱哪一种。

“线性”要求式子里只出现变量的一次项；系数是已知的常数。“整数”要求变量不能取 0.3 或 2.7。若进一步要求每个变量只能取 0 或 1，得到的就是 **0–1 整数规划**。可以把每个变量看成一只开关：开，还是关。

数独恰好非常适合这种开关语言。不过，先看一个容易想到却不好处理的写法。

3 为什么不用一个格子一个变量

给第 r 行、第 c 列的数字记一个变量 $y_{rc} \in \{1,\dots,9\}$，总共只有 81 个变量，似乎已经很简洁了。问题在于，“一行的数字互不相同”并不直接是一条线性等式或不等式。例如 $y_{r1} \neq y_{r2}$ 包含“小于或大于”两种可能，在线性整数规划里往往还得加辅助变量处理。

另一个诱惑是：一行九个数的和应当是 45，于是写

$$\sum_{c=1}^{9} y_{rc} = 45.$$

可惜它只管总和，管不了重复：一行全填 5，和也恰好是 45。事实上，整张棋盘全填 5 就满足所有行、列、宫的和约束，却显然不是数独（这里暂不加给定数）。

和对了，不等于规则对了

我们需要的是“每个数字恰好一次”，而不是“数字加起来碰巧一样”。在约束规划里可以直接使用 `AllDifferent` 这样的全异约束；要把问题写成**线性**整数规划，下面的指示变量更顺手。

4 关键换元：每个候选数都是一个开关

对标准九阶数独，定义

$$x_{rck} = \begin{cases} 1, & \text{第 } r \text{ 行第 } c \text{ 列填的是数字 } k, \\ 0, & \text{否则,} \end{cases} \qquad r,c,k \in \{1,\dots,9\}.$$

三个下标分别回答：哪一行、哪一列、哪个数字。总共有 $9^3 = 729$ 个二元变量。变量多了，规则却变得更好写了。

例如 $x_{127} = 1$ 表示第一行第二列填 7。同一格子的九只开关排成一列：

$$(x_{121}, x_{122}, \dots, x_{129}) = (0,0,0,0,0,0,1,0,0).$$

我们不是让格子“取一个数”，而是问九次“是不是这个数”，并要求九次回答里**恰有一次是“是”**。

从二维棋盘到三维开关

固定 k，把 x_{rck} 画成一张 9×9 的表，它标出了数字 k 在整张棋盘上的所有位置。九个数字就是九层表；每一层都要在每行、每列、每宫里各点亮一个位置，九层叠起来时，每个格子又只能亮一层。

5 四组规则，全部写成等式

5.1 每个格子恰好填一个数

固定位置 (r,c)，把数字下标 k 加起来：

$$\boxed{\sum_{k=1}^{9} x_{rck}=1 \qquad (\forall r,c).} \tag{1}$$

这给出 81 条等式。因为变量是 0 或 1，和为 1 就是恰好选一个。

5.2 每行的每个数字恰好出现一次

固定行 r 和数字 k，让列下标 c 走遍这一行：

$$\boxed{\sum_{c=1}^{9} x_{rck}=1 \qquad (\forall r,k).} \tag{2}$$

又是 81 条。比如 $\sum_c x_{rc7}=1$，说的是这一行恰好有一个 7。

5.3 每列的每个数字恰好出现一次

这次固定列 c 和数字 k，对行下标求和：

$$\boxed{\sum_{r=1}^{9} x_{rck}=1 \qquad (\forall c,k).} \tag{3}$$

依然是 81 条。

5.4 每宫的每个数字恰好出现一次

用 $a,b\in\{0,1,2\}$ 标出宫的位置，定义它的格子集合

$$B_{ab}=\{3a+1,3a+2,3a+3\}\times\{3b+1,3b+2,3b+3\}.$$

于是宫约束是

$$\boxed{\sum_{(r,c)\in B_{ab}} x_{rck}=1 \qquad (\forall a,b,k).} \tag{4}$$

九个宫，九种数字，还是 81 条。四组加起来共 324 条。**这是列出来的等式数，不是独立等式数**，其中有线性冗余。

最后，若题目给定 (r,c) 填 d，补上

$$x_{rcd}=1.$$

配合式 (1)，这个格子的其余八个开关自动为 0。若有 g 个给定数，直接写法就是 $324+g$ 条等式；实际实现也可以把这些变量的下界固定为 1，不增加矩阵的行数。

数独的完整模型

$$\begin{aligned}\min\quad & 0\\ \text{s.t.}\quad & \text{式 (1)–(4) 的四组规则},\\ & x_{rcd}=1\ \text{（对每一个给定数）},\\ & x_{rck}\in\{0,1\}.\end{aligned}$$

数独不需要在合法答案里比出高低，找到一个就行。所以目标函数取常数 0：**这是一个可行性问题。**也可以写成最小化 $\sum_{r,c,k}x_{rck}$，但这个和恒等于 81，没有筛选作用。

5.5 为什么这套翻译没有漏掉什么

给定一个合法数独，按它的数字点亮开关，四组规则和给定数约束显然成立。反过来，给定模型的一个二元可行解，格子约束保证每格能读出唯一一个数字；行、列、宫约束保证每个单位里各数字恰好一次；给定数约束保证题目未被改动。

因此，**合法数独与模型的二元可行解一一对应**。这比“计算机似乎算对了”更有力：模型没有凭空添答案，也没有丢答案。

6 回到四阶例子：方程怎样排除候选数

把所有求和上限换成 4，宫大小换成 2×2，模型就有 $4^3=64$ 个开关和 $4\cdot 4^2=64$ 条基本等式。来看第一行第二列。

第一行已有 1 和 4。由行约束与非负性，$x_{121}=x_{124}=0$；第二列已有 3，由列约束又有 $x_{123}=0$。格子约束于是只剩

$$0+x_{122}+0+0=1,$$

所以 $x_{122}=1$。它说的正是：第一行第二列填 2。随后第一行的数字 3 只能放到第三列。

这段排除推理与手算完全一致。不过，模型并没有把这套步骤写死：它只描述所有必须同时满足的条件，求解器自行决定先利用哪些条件。

7 为什么不能把整数条件扔掉

若把 $x_{rck}\in\{0,1\}$ 放宽成 $0\le x_{rck}\le 1$，就得到**线性松弛**。它更容易处理，却允许“半开半关”的格子。

以没有给定数的九阶棋盘为例，令所有 $x_{rck}=1/9$。每条基本等式都加了九个 $1/9$，恰好等于 1。因此它是松弛模型的可行解，却没有给出任何一个确定的数字。这不意味着求解器必然返回这组值，而是说明**线性等式本身没有排除它**。

还有一个更直观的证明：如果一道题有两个不同的合法解 $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}$，那么

$$\bar{\boldsymbol{x}} = \tfrac{1}{2}\boldsymbol{x}^{(1)} + \tfrac{1}{2}\boldsymbol{x}^{(2)}$$

满足同样的线性约束和给定数约束，但两个答案分歧的地方出现了 $1/2$。线性约束容得下“各选一半”，数独容不下。

分数不是候选数的概率

松弛里的 $x_{rck} = 1/2$ 只是一个连续变量值，不能直接解读成“这里有一半概率填 k”。分别四舍五入也不可靠：同一格可能全变成 0，一行也可能冒出两个相同数字。对有些题松弛已经足够，对另一些题仍要靠整数条件进一步搜索。

7.1 求解器在做什么

可以粗略理解为：先做预处理并利用松弛提供的信息；若还剩分数变量，就对某个开关分情况，例如 $x_{rck} = 0$ 或 1，分别继续求解。某个分支一旦与约束矛盾，就整枝排除。这就是**分支定界**的基本想法，实际求解器还会结合割平面等技术。

本题目标恒为 0，优化界在可行分支之间不提供高低之分；关键仍是排除不可能、找到整数可行解。写出 729 个开关，并不意味着真的枚举 2^{729} 种开关组合。

8 找到答案之后：它是唯一的吗

“找到一个解”与“只存在这一个解”是两件事。求解器第一次成功，只完成了前者。要验证后者，可以**禁掉已经找到的答案，再解一次**。

设第一次得到的棋盘是 s_{rc}，补上一条约束

$$\boxed{\sum_{r=1}^{9}\sum_{c=1}^{9} x_{rc,s_{rc}} \le 80.} \tag{5}$$

原答案在这 81 个位置上全部取 1，和等于 81，所以被排除。任意不同的棋盘至少改了一个格子，对应的和至多为 80，不会被误伤。这条约束通常叫**排除约束**（no-good cut）。四阶例子里右边换成 15。

第二次求解的三种结果

- **又找到一个整数可行解**：原题至少有两个解。
- **证明不可行**：原题只有第一次找到的那个解。
- **超时或异常终止**：目前无法判定唯一性。

如果第一次就证明不可行，则题目没有解，可能是抄错了给定数。“暂时没找到”不能代替“证明不存在”。

还可以不断禁掉已知答案来枚举解，但棋盘越空，解越可能很多。如果只想检查唯一性，找到第二个就可以停。

9 动手：用 Python 跑一遍

附录给出完整程序，另附同内容的 数独求解.py 文件。运行环境需要 NumPy 与 SciPy（SciPy 至少为 1.9，提供 `milp` 接口）。程序中的 `block=2` 表示 2×2 宫；换成 3，并输入 9×9 棋盘，就是标准数独。空格用 0 表示。

代码采用从 0 开始的下标，文章的下标从 1 开始，因此读取给定数时会把数字减 1。三维编号表 `index` 只是把位置 (r,c,k) 编号为一维变量，没有改变模型本身。

四阶题的输出应当是

```
[[1 2 3 4]
 [3 4 1 2]
 [2 1 4 3]
 [4 3 2 1]]
unique
```

`unique` 表示禁掉这个答案之后，第二次求解证明不可行。若输出 `multiple`，说明存在另一个答案；`infeasible` 则表示题目无解。程序把其他终止状态抛为异常，避免把计算中断误报为无解。

实现里先明确设置变量在 $[0,1]$ 之间，再设置整数性，二者合起来才是二元变量。只要求“整数”而忘了上界，并不能在一般问题里自动得到 0–1 开关。读取结果时也先检查它确实接近整数并满足约束，再恢复棋盘。

10 再往外走三步

10.1 精确覆盖：换一种语言看同一套模型

把四组基本约束写成 $A\boldsymbol{x}=\mathbf{1}$。矩阵 A 有 324 行、729 列，每一列对应一次候选放置 (r,c,k)。这一列恰好在四个位置为 1：占据哪个格子、占用哪行的哪个数字、占用哪列的哪个数字、占用哪宫的哪个数字。

于是问题变成：**选出一些候选列，让每一行恰好被覆盖一次**。这就是**精确覆盖**。给定数要求相应候选列必须被选中。它和整数规划用不同的算法语言描述同一件事；Donald Knuth 的 Algorithm X 及 Dancing Links 是求解精确覆盖的一条经典路线。

10.2 变体：改规则，也就是改约束

对角线数独要求两条长对角线上也各有 1 到 9。只需再加 18 条等式：

$$\sum_{r=1}^{9} x_{rrk}=1,\qquad \sum_{r=1}^{9} x_{r,10-r,k}=1\qquad (k=1,\dots,9).$$

如果某个笼子 C 要求格内数字和为 T，则补上

$$\sum_{(r,c)\in C}\sum_{k=1}^{9} k\,x_{rck}=T.$$

这里的 k 是系数，所以式子仍然是线性的。如果笼内还禁止重复，还得对每个 k 加 $\sum_{(r,c)\in C} x_{rck}\le 1$，单靠总和约束并不够。

10.3 出题：删除给定数，也需要检查唯一性

从一张完整棋盘出发，试着擦掉一个数字。每擦一次，都用“两次求解”的办法检查：剩下的线索是否仍确定唯一答案？如果出现多解，就把刚擦的数字放回去。

删到不能再删，不等于线索数最少

依次试删所有线索，可以得到一张**极小**题目：剩下的任一线索再删掉都会产生多解。但这不保证它的线索总数是所有题目中**最小**的，删除顺序也可能影响结果。而且“唯一”不等于“好玩”：人类解题难度、技巧分布和题面美感，需要另外评价。

11 小结：把技巧留给求解器，把规则写清楚

1. 用 $x_{rck} \in \{0,1\}$ 表示“这个位置是否填这个数”，把候选选择变成开关；变量增多，约束反而变简单。
2. 格子、行、列、宫四组“恰好一次”，加上给定数，完整刻画数独；九阶模型有 729 个变量、324 条基本等式。
3. 不必硬造一个要最大化的指标。数独是可行性问题，目标取 0 就够了。
4. 线性松弛可以帮助搜索，但分数解不能当答案，也不能靠逐项四舍五入保证合法。
5. 唯一性要单独检验：禁掉第一个答案之后，若能证明不可行，才说明唯一。

这套方法值得记住的地方，是它把“怎样填数”变成了“什么算填对”。在排班、任务指派、选址里也常能看到同样的动作：先把离散选择变成开关，再把不能违反的规则写成线性约束。

当然，通用整数规划求解器返回的答案未必附带适合人类阅读的数独推理链，也不保证在每道题上都比专用算法快。它的长处在于：规则写得清楚，模型便能检查答案、证明无解、检验唯一性，还允许我们从同一个起点，试着改变游戏。

参考与继续阅读

- MathWorks 官方示例，Solve Sudoku Puzzles via Integer Programming: Solver-Based。可对照阅读二元变量的数独表达。
- SciPy 官方文档，scipy.optimize.milp。附录程序的参数和终止状态以此接口为准。
- D. E. Knuth, *Dancing Links* (2000), arXiv:cs/0011047。了解精确覆盖与高效回溯的一条经典路线。

A　完整求解程序

以下代码同时完成第一次求解与唯一性检查，保存为 `数独求解.py` 即可运行。在已安装依赖的环境中执行 `python3 数独求解.py`。

A.1　构造变量与约束

```
"""Sudoku as a binary integer program. Requires NumPy and SciPy >= 1.9."""
import numpy as np
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import lil_matrix


def model(puzzle, block=2):
    p = np.asarray(puzzle)
    n = block * block
    if (p.shape != (n, n) or not np.issubdtype(p.dtype, np.integer)
            or np.any((p < 0) | (p > n))):
        raise ValueError("Expected an integer grid with entries 0..n")
    index = np.arange(n ** 3).reshape(n, n, n)
    groups = []
    for r in range(n):
        for c in range(n):
            groups.append(index[r, c, :])
    for r in range(n):
        for k in range(n):
            groups.append(index[r, :, k])
    for c in range(n):
        for k in range(n):
            groups.append(index[:, c, k])
    for br in range(0, n, block):
        for bc in range(0, n, block):
            for k in range(n):
                groups.append(index[br:br+block,
                                    bc:bc+block, k].ravel())
    A = lil_matrix((4 * n * n, n ** 3))
    for i, ids in enumerate(groups):
        A[i, ids] = 1
    lo, hi = np.zeros(n ** 3), np.ones(n ** 3)
    for r, c in zip(*np.nonzero(p)):
        lo[index[r, c, p[r, c] - 1]] = 1
    return index, Bounds(lo, hi), LinearConstraint(A.tocsc(), 1, 1)
```

A.2 求解、排除原答案并再次求解

```
def solve(puzzle, block=2):
    index, bounds, rules = model(puzzle, block)
    n = block * block
    def run(constraints):
        return milp(c=np.zeros(n ** 3),
                    integrality=np.ones(n ** 3),
                    bounds=bounds, constraints=constraints)
    first = run([rules])
    if first.status == 2:
        return None, "infeasible"
    if first.status != 0:
        raise RuntimeError(first.message)
    bits = np.rint(first.x)
    assert np.max(np.abs(first.x - bits)) < 1e-6
    assert np.allclose(rules.A @ bits, 1)
    assert np.all(bits >= bounds.lb) and np.all(bits <= bounds.ub)
    grid = bits.reshape(n, n, n).argmax(axis=2) + 1
    cut = lil_matrix((1, n ** 3))
    cut[0, np.flatnonzero(bits)] = 1
    exclude = LinearConstraint(cut.tocsc(), -np.inf, n * n - 1)
    second = run([rules, exclude])
    if second.status == 2:
        return grid, "unique"
    if second.status == 0:
        return grid, "multiple"
    raise RuntimeError(second.message)


if __name__ == "__main__":
    puzzle = [[1, 0, 0, 4], [0, 4, 1, 0],
              [2, 0, 4, 0], [0, 3, 0, 1]]
    grid, kind = solve(puzzle)
    print(grid)
    print(kind)
```